CS 64 HANDOUT #1

Numerical Bases, Binary Addition and Logic Operations

Prof. Ziad Matni

*Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture!* ☺

In this reading, we will discuss and clarify:

- Numerical bases and conversions between different bases.
- The Two's Complement method to express negative binary numbers.
- Binary addition and logic operations.
- Example of use of binary operations in the C++ programming language.

1. Numerical Bases

We normally (i.e., in our time in history and in our general culture) count in 10s.
That is, if we express a number like **672**, we mean: **6 hundreds** (i.e., 6 x 10^2), **7 tens** (i.e., 7 x 10^1) and **2 units** (i.e., 2 x 10^0). This is called counting in ***decimal base***, or in ***Base-10***. In Base-10 we express numbers using 10 digits (that's not a coincidence): 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.

However, we could count in ANY numerical base (historically, other cultures have done that)! This is easy to do once you keep track of the *position* of each digit. ***Positional notation*** (see https://en.wikipedia.org/wiki/Positional_notation) is an easy way to allow you to *switch (i.e., **convert**)* between any base and Base-10.

For example: In Base-2 (i.e., binary), where we express numbers with 2 digits, 0 and 1, the number: **101101**
is really:

$$\mathbf{1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0}$$

Note that the position of the digit is represented in the *power of the base* (the base is 2 in this example). In case you did not know this: we call binary digits ***bits***. Note also that the left-most position (also known as the ***least-significant digit***) is ***position zero***. As it happens, the number above calculates to: 32 + 8 + 4 + 1 = 45 (as expressed in decimal base).

In Computer Science/Engineering, it is important to be "conversant" in Base-2 and its related numerical bases, like **Base-8 (octal)** and, more commonly, **Base-16 (hexadecimal)**. It's also very helpful to know the powers of 2 and recall them quickly.

The number in our previous example, **101101** (binary), can be expressed very easily in octal by realizing that each 3 binary *bits* are a collection that can range from 0 to 7 (i.e., the digit range for Base-8/octal).

So, 101101 (Base-2) → [101][101] (I collected every 3 bits together, starting from the right)
When expressed as a digit in Base-8, that's **55** (since 101 is binary for 5).
In other words, **101101 in Base-2 = 55 in Base-8** (and, as previously noted, 45 in Base-10)

Likewise, *collecting 4 binary bits* in a Base-2 number can help us convert a Base-2 to a Base-16 number. So, 101101 (Base-2) → [10][1101] (I collected every 4 bits together, starting from the right – also known as from the *least significant bit* first).
When expressed as digits in Base-16, that's **2D**.
In other words, **101101 in Base-2 = 2D in Base-16**.

In case you're wondering what that "2D" number is, you may recall from other CS/Math courses, that Base-16/hexadecimal uses 16 digits to express a number. The 2 tables below are good ones to know very intimately throughout your CS/CE career:

Power of 2	Value
2^1	2
2^2	4
2^3	8
2^4	16
2^5	32
2^6	64
2^7	128
2^8	256
2^9	512
2^{10}	1024

Hex. Digit	4-bit Value
0x0	0000
0x1	0001
0x2	0010
0x3	0011
0x4	0100
0x5	0101
0x6	0110
0x7	0111

Hex. Digit	4-bit Value
0x8	1000
0x9	1001
0xA (10)	1010
0xB (11)	1011
0xC (12)	1100
0xD (13)	1101
0xE (14)	1110
0xF (15)	1111

Note that hexadecimal numbers are, by convention, written with a **0x** prefix. This does not have any value whatsoever – it only indicates that the number following is expressed in hexadecimal.

From now on, we will utilize this convention when referring to hexadecimal numbers.

2. Converting from Decimal to Other Numerical Bases

When converting from decimal to any other base, *N*, the simplest algorithm is the following (let's call it the Division Algorithm):

> *While (the quotient is not zero) do this:*
> *Divide the decimal number by N and find the **remainder***
> *Make that remainder the **next digit to the left** in the answer*
> *Replace the original decimal number with the **quotient***
> *Repeat until your **quotient is zero***

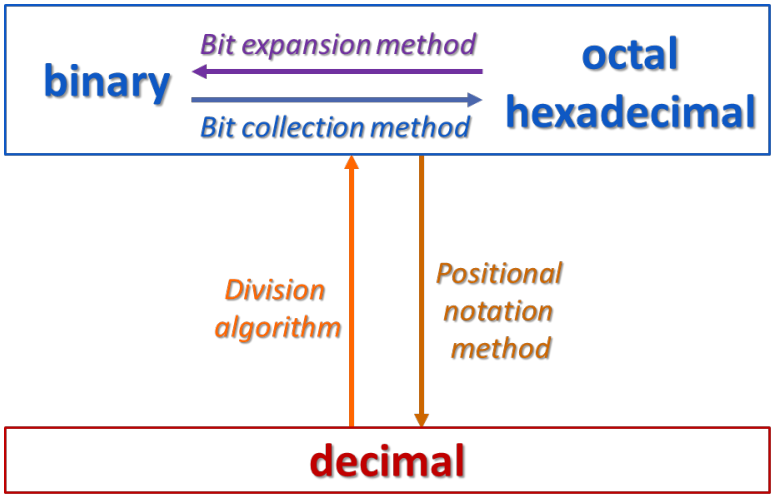
Here is an example to illustrate: Suppose you want to convert the decimal number **98** to octal (i.e., Base-8):

98 / **8** = 12 R **2** *Note: the new base is **8**, the remainder is **2**, the quotient is **12**.*
12 / **8** = 1 R **4**
1 / **8** = 0 R **1** *Note: the quotient is now zero, so the algorithm ends here.*

Recall that each remainder is placed to the left of the previously found remainders. In other words, the decimal number **98**, when converted into octal is **142**.

You can double-check your work by using positional notation on your answer:
142 (octal) = $1 \times 8^2 + 4 \times 8^1 + 2 \times 8^0 = 64 + 32 + 2 = 98$ (decimal).

The following diagram illustrates the 4 methods of numerical conversion that we use:



3. Two's-Complement Method for Expressing Negative Binary Numbers

Two's complement is a mathematical operation that allows us to convert a *positive* binary number into its *negative value*. It is not the only way to do this sort of operation, but it is used *universally* in Computer Science/Engineering and computer hardware is built on this basis (so, computer software has to use this method as well).

The process is as follows:

1. Start with your positive binary number, e.g., **0101** (that's **+5** as expressed in 4 bits).
2. *Reverse each bit*, that is, make each 0 a 1, and 1 a 0. This makes our example: **1010**.
3. *Now add 1* to this result: 1010 + 1 = **1011**.

The binary number **1011** now represents **-5** in 4 bits.

The beauty of the 2's complement method is that it is *reversible* – meaning, you can use the *same* algorithm to also go from negative to positive! Observe:

1. Start with your ***negative*** binary number from the last example, **1011**, which we ascertained was -5.
2. *Reverse each bit*. This makes our example: **0100**.
3. *Now add 1* to this result: 0100 + 1 = **0101 – which is the number +5 *(!!!)*** 😮

Note that, when expressing a number in 2's complement, it's easy to tell what number is a positive number and what number is negative – can you guess what that is?! Along those lines, think about how you can quickly tell if a binary number is an even or odd number.

The number of bits that we use to express a 2's complement number is very important. Take, for example, the same number we started with (+5) but express it in 8 bits: 00000101. The 2's complement becomes: 11111011.

That's because the number of bits we chose to represent a binary number gives us *the permitted range*. In pure mathematics, this may not be a very interesting point, but in Computer Science/Engineering, this is of ***critical importance!*** *That's because when you represent any number inside of a computer, you have a limited number of bits in which to do so (computers are physical machines, after all).*

Generally speaking, if you have **N** bits to represent a purely positive number (i.e., an ***unsigned integer***), the range of numbers you can express go from **0 to $2^N - 1$** (hence, a total of 2^N numbers). So, when using 8-bit unsigned numbers, for example, your range is from 0 to 255.

However, if you have **N** bits to represent a number that can be positive *or* negative (i.e., a ***signed integer***), the range of numbers you can express go from **-2^{N-1} to $+2^{N-1} - 1$** (again, note how that's a total of 2^N numbers). So, when using 8-bit *signed numbers*, for example, your range is from -128 to +127.

4. Carry Out and Overflow

When adding 2 binary numbers, the addition takes place just like adding 2 decimal numbers – the same way you learned it in your first year of elementary school…!
For example, take **0110 + 1111**:

```
   11   ← carry overs
  0110
+ 1111
= 1 0101
```

If you want to think about it in purely mathematical terms, what we just did was add 6 (0110) to 15 (1111) and so got 21 (10101).

However, in CS/CE terms, we note *intently* that we added two 4-bit numbers and ended up with an extra 5th bit at the leftmost end because of the carrying-over. In a computer, if a value has a set size (for example, integers in most programming languages are assigned 32 bits), then all operations on it need to result in a binary value *of that same set size*. Remember, a computer is a *physical machine* and not a magical mathematical being!!

In terms of ***unsigned integers***, that extra 5th bit in our last example is called a **Carry Out Bit** (often denoted with a **C**) and, when equal to 1, it indicates that our sum lies *outside the scope of the range* of the 4-bit numbers we were working with. Note that with 4-bits, the range of numbers in this instance, is 0 to 15. Our answer, 17, is definitely outside that range! This is a potential error for a computer to have to deal with (if said computer was only using 4-bits to express unsigned integers – not a realistic scenario, but it'll do for illustrative purposes).

NOW: If the 2 numbers we just added were actually ***signed integers***, note that we just really added +6 (0110) to -1 (1111) and so got +5 (0101 – just looking at the 4 bits, not that 5th extra bit on the left-most side). If this is the case, then we have to concern ourselves with a *different* indicator than the Carry Out Bit to tell us if our answer lies outside the scope of the *signed* range of the 4-bit numbers we were working with. Note that in our example, +6 - 1 got us +5, which is inside the range of these *signed integers* (that range, using 4-bits, would be -8 to +7). So, there's no potential error there. But what's the indicator that definitively tells us this?

The indicator we're looking for with ***signed integers*** is called the **Overflow Bit** (often denoted with a **V**) and, if it's equal to 1, it shows that our sum lies *outside the scope of the range* of the 4-bit numbers we were working with. ***V is not the same as C!!!***

The rule for figuring out if we have overflow (i.e., if **V** = 1) is as follows:

When adding 2 n-bit ***signed integers***: **x** + **y** and we end up with the answer **s** (also n-bits)
if **x**, **y** > 0 AND **s** < 0
OR if **x**, **y** < 0 AND **s** > 0
Then, **overflow** has occurred (i.e., that indicator bit for overflow, **V** = 1).

Again, Carry Out and Overflow are 2 independent bits and we only *care* to know what the Carry Out Bit is when we are adding ***unsigned integers*** (since Overflow makes "no sense" with unsigned integers) and we only *care* to know what the Overflow Bit is when we are adding ***signed integers*** (since Carry Out makes "no sense" with signed integers).

5. Binary Logic Refresher

inputs *output*

X	Y	X AND Y X && Y X.Y
0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

input *output*

X	NOT X $\overline{X}$
0	**1**
1	**0**

inputs *output*

X	Y	X OR Y X | | Y X + Y
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**1**

X	Y	X XOR Y X ⊕ Y
0	0	**0**
0	1	**1**
1	0	**1**
1	1	**0**

In this course, we will be referring to the binary (i.e., Boolean) logic operators AND, OR, NOT and XOR. You should have seen these in a prior CS/Logic course. Their truth tables are shown above.

We can use these logic operators on any integer on a *bit-by-bit basis.* This is called using ***bitwise logic operators***.

For example, suppose I wanted to do the **bitwise-AND** operation on 0110 and 1011, I would get:

```
   0110
&  1011
=  0010
```

Another example: perform the **bitwise-OR** operation on 0x51 and 0xF2:

```
   0101 0001
|  1111 0010
=  1111 0011
   = 0xF3
```

The *operator symbols* for these bitwise operations are shown here and can be used in C/C++ programming:

&	bitwise-AND
|	bitwise-OR
^	bitwise-XOR
~	bitwise-NOT

Note that *Boolean* logic operators in C++ programming are *different* from bitwise logic operators (i.e., && means Boolean AND, || means Boolean OR).

6. Bit-Shifting Binary Operations

Bit-Shifting is a commonly used operation on binary numbers in computers.
It means moving (shifting) all the bits ***N*** positions to the left (or to the right).

For example:

Bit Shift Left operation *(C/C++ operator: **<<**)*

note: this is NOT the same operator used with std::cout!

1. Bit shift left by 2 places, the binary number 1001:
   1001 << 2 = 1001**00**
   Note how we put in zeros in the places left blank by the bit-shifting

2. Bit shift left by 4 places, the binary number 11011:
   11011 << 4 = 11011**0000**

Note that bit-shifting to the left does a *certain type of* multiplication to the binary number.
As an exercise, think about and explain how that works.

You can also bit-shift to the *right*:
For example:

Bit Shift Right operation *(C/C++ operator: **>>**)*

note: this is NOT the same operator used with std::cin!

3. Bit shift **right** by 2 places, the binary number 1001:
   1001 >> 2 = 10
   Note how we dropped the 2 right-most bits with this operation

4. Bit shift right by 4 places, the binary number 11011:
   11011 >> 4 = 1

Note that bit-shifting to the right does a *certain type of* division on the binary number.
As an exercise, think about and explain how that works.

7. C++ Example

Consider this C++ function:

```
void fx (int n) {
      n = n >> 3;
      n = n - 2;
      n = n & 6;
      std::cout << n << std::endl;
} // end fx
```

If you called the function as **`fx(40)`**, what would happen?
Again, recall that C++ **int** types are 32-bits long.

The answer is that you'll see the number **2** printed out. The reason is as follows:

If **n = 40** (which, in 32-bits, is 00000000000000000000000000101000 – let's ignore all those leading zeros and easily see that **101000** is indeed 32 + 8, i.e., 40).

We first bit-shift **n** to the *right* by 3 bits, so **n** = **101000** becomes **n** = **101** in binary. Note that this is the decimal number 5, by the way.

Then we subtract 2 from **n**, so **n** becomes 3 (i.e., **n** = **011** in binary).

Finally, we bitwise-AND **n** to 6 (i.e., to **110** in binary). This gives us **n** = **010** in binary (can you figure out how that worked?), which is the final answer.

When the **cout** statement is executed, we'll see the number **2** displayed on the screen!
Go ahead and try running this function in a C++ program! ☺

As an exercise, think about how you might use a similar C++ program to tell you if an integer's binary bit at the 7th position (with the right-most bit being position zero) is a 1 or a 0. For example, consider the (decimal) integer **1,423**, which has the 32-bit binary value:
00000000000000000000010110001111.
The bit in position 7 is illustrated in blue above and underlined (it's a 1).

One way you could do this is to take the integer value, bit shift it to the right by 7 places (so the bit in position 7 becomes in position 0) and then bitwise-AND that result with 1.
Think about why you might do that: what does that do?